Amundsen.io

RESUMEN DE LAS EXPERIENCIAS Y EVIDENCIAS DE LA INVESTIGACIÓN

C. Enrique Israel Torres Mendoza

Dirección del Laboratorio de Ciencia de Datos y Métodos Modernos de Producción de Información

2023

Introducción.

En la presentación se pretende documentar un resumen de los avances en la investigación e implementación de la herramienta Amundsen.io, este motor se implementa en un sistema operativo tipo Linux, específicamente en una imagen de Red Hat 8.7, el cual también está instalado en una máquina virtual en la herramienta VMWARE Workstation.

El autor documenta la investigación y da su opinión de los recursos disponibles para la investigación e implementación de Amundsen y da sus conclusiones hasta la fecha en el avance del presente proyecto.

Amundsen cuenta con varios componentes como son:

- Frontend
- Search
- Metadata
- Databuilder
- Common

En el presente avance se ha investigado hasta el componente de metadatos.

Amundsen.io

Vemos en la documentación de Amundsen.io su definición y los elementos que componen esta herramienta.

Amundsen es un motor de metadatos y descubrimiento de datos para mejorar la productividad de los analistas de datos, los científicos de datos y los ingenieros cuando interactúan con los datos. Lo hace hoy mediante la indexación de recursos de datos (tablas, tableros, flujos, etc.) y potenciando una búsqueda de estilo de rango de página basada en patrones de uso (por ejemplo, las tablas más consultadas aparecen antes que las tablas menos consultadas). Piense en ello como una búsqueda de datos en Google. El proyecto lleva el nombre del explorador noruego Roald Amundsen, la primera persona en descubrir el Polo Sur.

Amundsen está alojado en LF AI & Data Foundation. Incluye tres microservicios, una biblioteca de ingesta de datos y una biblioteca común.

Frontend service

Para usar Amundsen es necesario instalar su servicio de frontend

El servicio frontend aprovecha un servicio de búsqueda independiente y un servicio de metadatos independiente para ver y editar metadatos para un recurso determinado.

Está hecho en React.

Los requisitos para Amundsen frontend son los siguientes:

Amundsen
Overview
Architecture
Developer Guide
User Guide
Components
Frontend
Search
Metadata
Databuilder
Common
FAQ
Roadmap

Requirements

- Python >= 3.6
- Node = v10 or v12
- npm >= 6.x.x

En nuestra maquina virtual con Red Hat en modo superusuario verificamos que hasta la fecha ya tengamos instalados en el sistema los requerimientos de Amundsen.io

Las siguientes instrucciones son para configurar una versión independiente de la aplicación Amundsen. Este enfoque es ideal para el desarrollo local.

```
[root@localhost sirilo]# python3 --version
Python 3.6.3
[root@localhost sirilo]# node --version
v10.24.0
[root@localhost sirilo]# npm --version
6.14.11
[root@localhost sirilo]#
```

https://www.amundsen.io/amundsen/frontend/

Instalación.

Las siguientes instrucciones son para configurar una versión independiente de la aplicación Amundsen. Este enfoque es ideal para el desarrollo local.

```
# Clone repo
$ git clone https://github.com/amundsen-io/amundsen.git

# Build static content
$ cd amundsen/frontend/amundsen_application/static
$ npm install
$ npm run build # or npm run dev-build for un-minified source
$ cd ../../

# Install python resources
$ python3 -m venv venv
$ source venv/bin/activate
$ pip3 install -e ".[all]" .

# Start server
$ python3 amundsen_application/wsgi.py
# visit http://localhost:5000 to confirm the application is running
```

https://www.amundsen.io/amundsen/frontend/docs/installation/

Instalación.

Para el caso del presente entorno, se descarga la aplicación de Amundsen frontend con la ayuda de la herramienta Podman con el siguiente comando.

```
podman pull docker.io/amundsendev/amundsen-frontend:latest
```

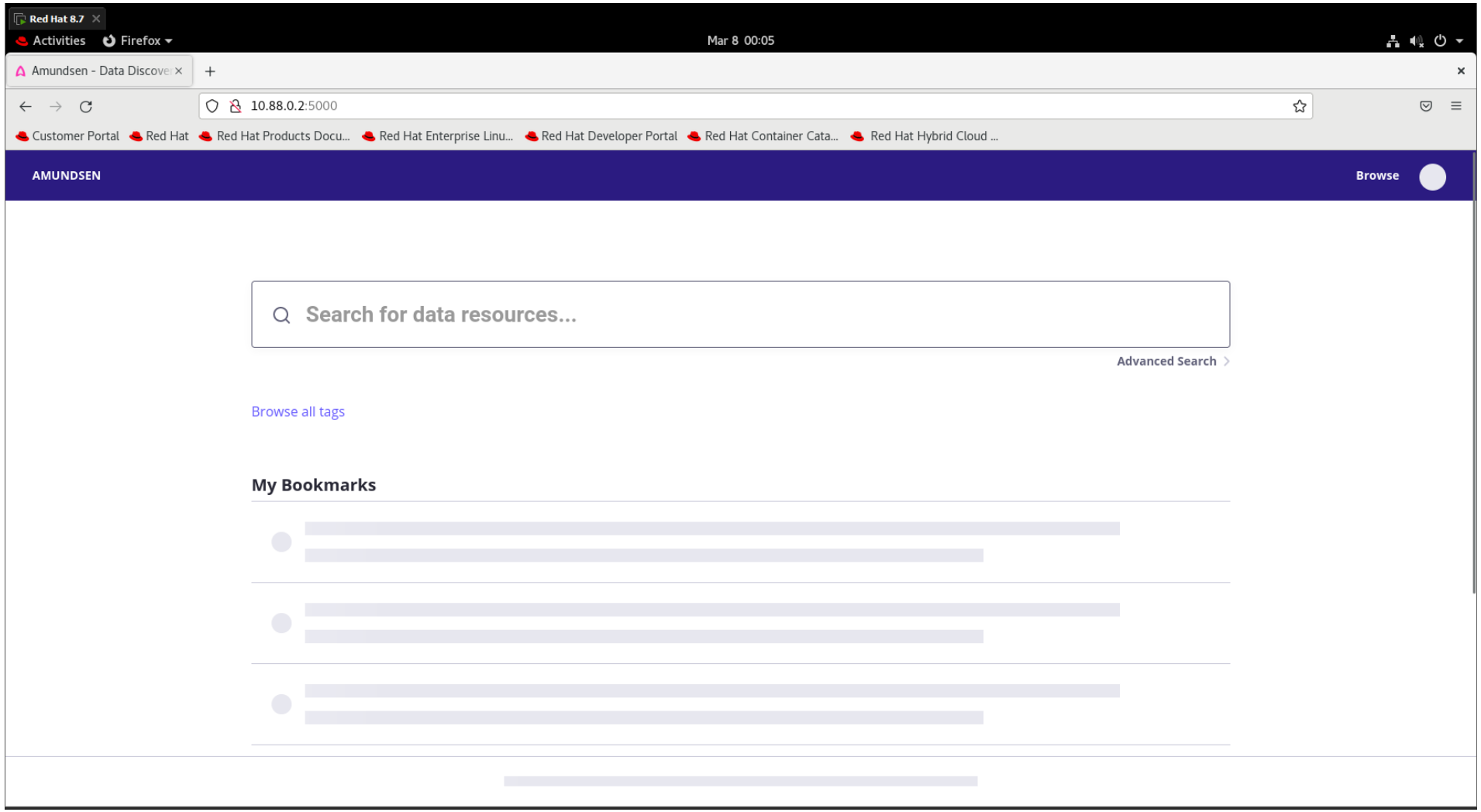
```
[root@localhost sirilo]# podman pull docker.io/amundsendev/amundsen-frontend
Trying to pull docker.io/amundsendev/amundsen-frontend:latest...
Getting image source signatures
Copying blob 31078b2538a2 skipped: already exists
Copying blob f63e77b7563a skipped: already exists
Copying blob 3b6aeef29d24 skipped: already exists
Copying blob 51a05345c44d skipped: already exists
Copying blob dca49bd08fde skipped: already exists
Copying blob 42c077c10790 skipped: already exists
Copying blob e69ebd661d90 skipped: already exists
Copying blob 52af7a8c1c65 skipped: already exists
Copying blob 8f2af027defb skipped: already exists
Copying blob d518e496c9c4 skipped: already exists
Copying blob 624feb6228ab skipped: already exists
Copying blob 67191f138b1f skipped: already exists
Copying config 88ea40995e done
Writing manifest to image destination
Storing signatures
88ea40995e31ca78511a2c3019c1c174d31146467351c86b41be99993fc3d1a4
[root@localhost sirilo]#
```

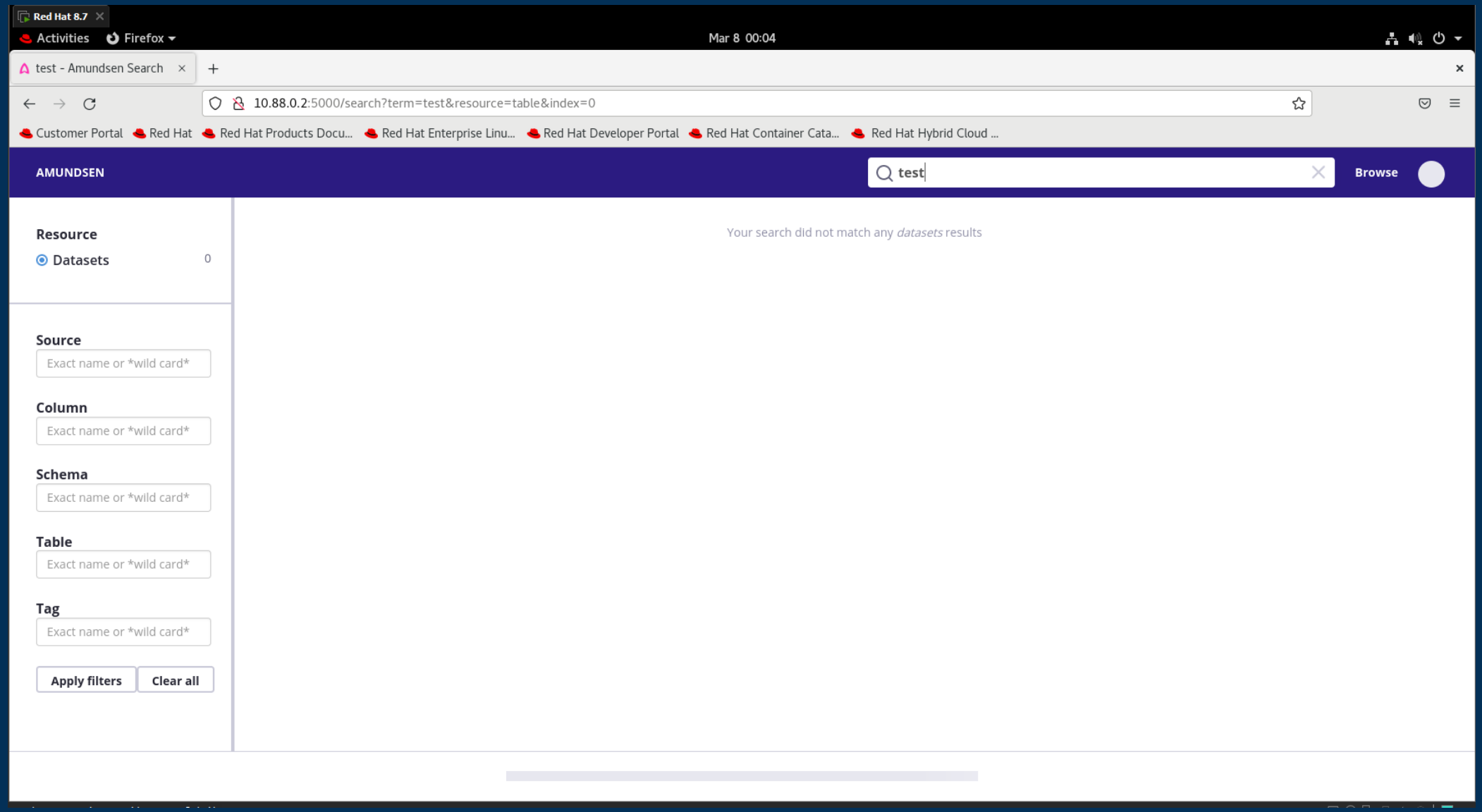
Posteriormente se ingresa el siguiente comando para crear y levantar el contenedor del servicio de frontend de Amundsen.

```
podman run -it --rm -p 8080:8080 --name a_front docker.io/amundsendev/amundsen-frontend
```

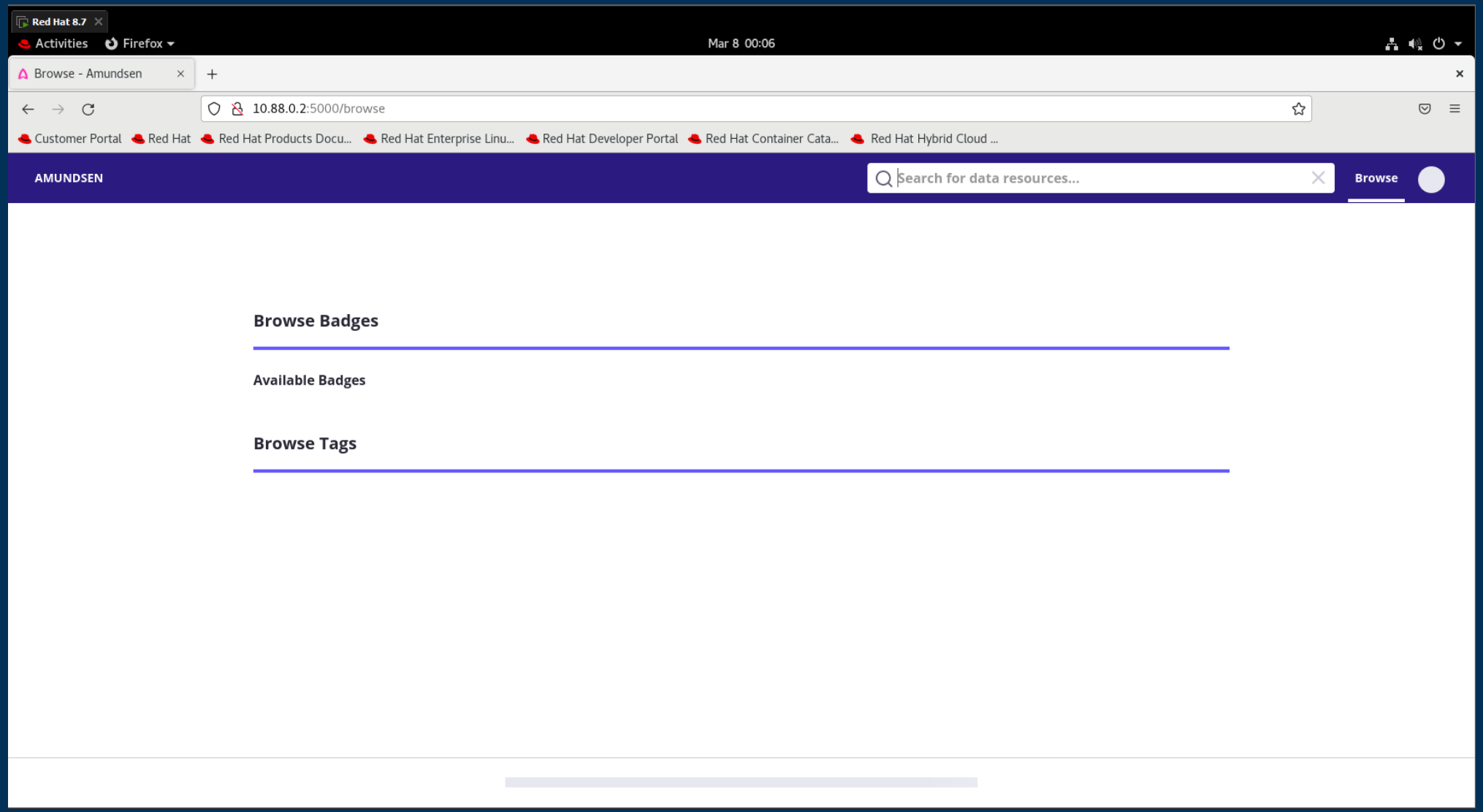
```
[root@localhost static]# podman run -it --rm -p 8080:8080 --name a_front docker.io/amundsendev/amundsen-frontend
Registering post_exec action_log entry_point: logging_action_log = amundsen_application.log.action_log_callback:logging_action_log
2023-03-08T04:48:42+0000.695 [INFO] __init__.create_app:69 (1:MainThread) - Created app with config name amundsen_application.config.TestConfig
2023-03-08T04:48:42+0000.695 [INFO] __init__.create_app:70 (1:MainThread) - Using metadata service at http://0.0.0.0:5002
2023-03-08T04:48:42+0000.695 [INFO] __init__.create_app:71 (1:MainThread) - Using search service at http://0.0.0.0:5001
 * Serving Flask app "amundsen_application" (lazy loading)
 * Environment: production
   WARNING: Do not use the development server in a production environment.
   Use a production WSGI server instead.
 * Debug mode: off
2023-03-08T04:48:42+0000.740 [WARNING] _internal._log:225 (1:MainThread) -  * Running on all addresses.
   WARNING: This is a development server. Do not use it in a production deployment.
2023-03-08T04:48:42+0000.741 [INFO] _internal._log:225 (1:MainThread) -  * Running on http://10.88.0.2:5000/ (Press CTRL+C to quit)
```

Finalmente abrimos el enlace donde se está ejecutando el servicio




Red Hat 8.7
Activities
Firefox
Mar 8 00:04
test - Amundsen Search
10.88.0.2:5000/search?term=test&resource=table&index=0
Customer Portal
Red Hat
Red Hat Products Docu...
Red Hat Enterprise Linu...
Red Hat Developer Portal
Red Hat Container Cata...
Red Hat Hybrid Cloud ...
AMUNDSEN
test
Browse
Resource
Datasets
0
Source
Exact name or *wild card*
Column
Exact name or *wild card*
Schema
Exact name or *wild card*
Table
Exact name or *wild card*
Tag
Exact name or *wild card*
Apply filters
Clear all
Your search did not match any datasets results

INEGI


Red Hat 8.7
Activities
Firefox
Mar 8 00:06
Browse - Amundsen
10.88.0.2:5000/browse
Customer Portal
Red Hat
Red Hat Products Docu...
Red Hat Enterprise Linu...
Red Hat Developer Portal
Red Hat Container Cata...
Red Hat Hybrid Cloud ...
AMUNDSEN
Search for data resources...
Browse
Browse Badges
Available Badges
Browse Tags

INEGI

Search service

Requirements

- Python >= 3.7
- Elasticsearch, supported versions:
  - 7.x
  - 8.0.0

En nuestra maquina virtual con Red Hat en modo superusuario verificamos que hasta la fecha tengamos una versión adecuada de Python y también una versión soportada de Elasticsearch.

Para poder hacer uso del servicio de frontedn de Amundsen es necesario montar el servicio de búsqueda de Amundsen (Amundsen Search service)

El servicio de búsqueda de Amundsen sirve una API Restful y es responsable de buscar metadatos. El servicio aprovecha Elasticsearch para la mayoría de sus capacidades de búsqueda.

```
[root@localhost sirilo]# systemctl status elasticsearch.service
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; enabled; vendor preset: disabled)
   Active: active (running) since Tue 2023-03-07 23:28:56 EST; 43min ago
     Docs: http://www.elastic.co
 Main PID: 1533 (java)
    Tasks: 38 (limit: 11058)
   Memory: 96.1M
   CGroup: /system.slice/elasticsearch.service
           └─1533 /bin/java -Xms1g -Xmx1g -XX:+UseConcMarkSweepGC -XX:CMSInitiatingOccupancyFractior

Mar 07 23:28:56 localhost.localdomain systemd[1]: Started Elasticsearch.

[root@localhost sirilo]# systemctl version elasticsearch.service
Unknown operation version.
```

```
[root@localhost sirilo]# python3 --version
Python 3.6.3
```

https://www.amundsen.io/amundsen/search/

Instalación.

Las siguientes son instrucciones para iniciar el servicio por Docker

```
$ docker pull amundsendev/amundsen-search:latest
$ docker run -p 5001:5001 amundsendev/amundsen-search
# - alternative, for production environment with Gunicorn (see its homepage link below)
$ ## docker run -p 5001:5001 amundsendev/amundsen-search gunicorn --bind 0.0.0.0:5001 search_service.search_wsgi

# In a different terminal, verify the service is up by running
$ curl -v http://localhost:5001/healthcheck
```

Para la presente practica se pretende sustituir Docker por Podman con lo cual los comandos para iniciar el servicio serían los siguientes:

```
podman pull docker.io/amundsendev/amundsen-search:latest
podman run -it --rm -p 5001:5001 --name a_search docker.io/amundsendev/amundsen-search
```

https://www.amundsen.io/amundsen/search/

```
[root@localhost sirilo]# podman pull docker.io/amundsendev/amundsen-search:latest
Trying to pull docker.io/amundsendev/amundsen-search:latest...
Getting image source signatures
Copying blob 9451cb82a066 skipped: already exists
Copying blob fa7d81b69b9a skipped: already exists
Copying blob 7c8f45ffebd6 skipped: already exists
Copying blob bd5d81492901 skipped: already exists
Copying blob 214ca5fb9032 skipped: already exists
Copying blob 3ce2a8d9fbf4 skipped: already exists
Copying blob 2fd9be6893d2 skipped: already exists
Copying blob 13df8ba24fdb skipped: already exists
Copying blob e46f3231827b skipped: already exists
Copying blob c33d07c2d600 skipped: already exists
Copying blob fd1f7c226ef0 skipped: already exists
Copying config e443c6b7e6 done
Writing manifest to image destination
Storing signatures
e443c6b7e6096065ce8395fda0b1e1665cbb5f7a267ee275603b203a197dbca9
[root@localhost sirilo]#
```

```
[root@localhost sirilo]# podman run -it --rm -p 5001:5001 --name a_search docker.io/amundsendev/amundsen-search
2023-03-08T05:09:10+0000.959 [INFO] __init__.create_app:85 (1:MainThread) - Creating app with config name search_service.config.LocalConfig
2023-03-08T05:09:10+0000.959 [INFO] __init__.create_app:86 (1:MainThread) - Created app with config name search_service.config.LocalConfig
 * Serving Flask app "search_service" (lazy loading)
 * Environment: production
   WARNING: Do not use the development server in a production environment.
   Use a production WSGI server instead.
 * Debug mode: off
2023-03-08T05:09:11+0000.047 [WARNING] _internal._log:225 (1:MainThread) -  * Running on all addresses.
   WARNING: This is a development server. Do not use it in a production deployment.
2023-03-08T05:09:11+0000.047 [INFO] _internal._log:225 (1:MainThread) -  * Running on http://10.88.0.3:5001/ (Press CTRL+C to quit)
```

Metadata service

Para poder hacer uso del servicio de frontedn de Amundsen es necesario también montar el servicio de metadatos (Amundsen metadata service)

El servicio de metadatos de Amundsen sirve a la API Restful y es responsable de proporcionar y actualizar los metadatos, como la descripción de la tabla y la columna, y las etiquetas. El servicio de metadatos puede utilizar Neo4j, Apache Atlas, AWS Neptune o Mysql RDS como capa persistente.

Requirements

- Python >= 3.7

```
[root@localhost sirilo]# python3 --version
Python 3.6.3
```

En nuestra maquina virtual con Red Hat en modo superusuario verificamos que hasta la fecha tengamos una versión adecuada de Python para el servicio de metadatos de amundsen

https://www.amundsen.io/amundsen/metadata/

Instalación.

Las siguientes son instrucciones para iniciar el servicio de metadatos mediante Docker

```
$ docker pull amundsendev/amundsen-metadata:latest
$ docker run -p 5002:5002 amundsendev/amundsen-metadata
# - alternative, for production environment with Gunicorn (see its homepage link below)
$ ## docker run -p 5002:5002 amundsendev/amundsen-metadata gunicorn --bind 0.0.0.0:5002 metadata_service.metadata_wsgi

-- In a different terminal, verify getting HTTP/1.0 200 OK
$ curl -v http://localhost:5002/healthcheck
```

Para la presente practica se pretende sustituir Docker por Podman con lo cual los comandos para iniciar el servicio serían los siguientes:

```
podman pull docker.io/amundsendev/amundsen-metadata:latest
podman run -it --rm -p 5002:5002 --name a_metadata docker.io/amundsendev/amundsen-metadata
```

https://www.amundsen.io/amundsen/metadata/

```
[root@localhost sirilo]# podman pull docker.io/amundsendev/amundsen-metadata:latest
Trying to pull docker.io/amundsendev/amundsen-metadata:latest...
Getting image source signatures
Copying blob 3df8cc82bc71 skipped: already exists
Copying blob e2be974225ed skipped: already exists
Copying blob b85a868b505f skipped: already exists
Copying blob 2477d560ded2 skipped: already exists
Copying blob ab9320c79a6c skipped: already exists
Copying blob 6e8c19ac0f87 skipped: already exists
Copying blob a97a867c82b4 skipped: already exists
Copying blob e9c97c7d2228 skipped: already exists
Copying blob 923f54a9d578 skipped: already exists
Copying blob be7ef3df33db skipped: already exists
Copying blob 917e60f00b1a skipped: already exists
Copying blob 8cf57fe60d24 skipped: already exists
Copying blob a9658566b9a3 skipped: already exists
Copying blob a912cfffce52 skipped: already exists
Copying config 356106bdc9 done
Writing manifest to image destination
Storing signatures
356106bdc956f22694f3bcf415d051d658d641fe442caef211f15f6740f1b1c7
[root@localhost sirilo]#
```

```
[root@localhost sirilo]# podman run -it --rm -p 5002:5002 --name a_metadata docker.io/amundsendev/amundsen-metadata
WARNING:root:amundsen_gremlin not installed. GremlinConfig and NeptuneConfig classes won't be available!
    Please install amundsen-metadata[gremlin] if you desire to use those classes.

INFO:root:Created app with config name metadata_service.config.LocalConfig
INFO:root:Using backend metadata_service.proxy.neo4j_proxy.Neo4jProxy
 * Serving Flask app "metadata_service" (lazy loading)
 * Environment: production
   WARNING: Do not use the development server in a production environment.
   Use a production WSGI server instead.
 * Debug mode: on
WARNING:werkzeug: * Running on all addresses.
   WARNING: This is a development server. Do not use it in a production deployment.
INFO:werkzeug: * Running on http://10.88.0.4:5002/ (Press CTRL+C to quit)
INFO:werkzeug: * Restarting with stat
WARNING:root:amundsen_gremlin not installed. GremlinConfig and NeptuneConfig classes won't be available!
    Please install amundsen-metadata[gremlin] if you desire to use those classes.

INFO:root:Created app with config name metadata_service.config.LocalConfig
INFO:root:Using backend metadata_service.proxy.neo4j_proxy.Neo4jProxy
WARNING:werkzeug: * Debugger is active!
INFO:werkzeug: * Debugger PIN: 508-698-319
```

Apache Atlas

El servicio de metadatos de Amundsen puede utilizar Apache Atlas como backend. Algunos de los beneficios de usar Apache Atlas en lugar de Neo4j es que Apache Atlas ofrece complementos para varios servicios (por ejemplo, Apache Hive, Apache Spark) que permiten actualizaciones basadas en inserción. También permite establecer políticas sobre qué metadatos son accesibles y editables mediante Apache Ranger.

Apache Atlas es un servicio de gobierno de datos, lo que significa que también obtiene la interfaz de usuario de Apache Atlas para acceder fácilmente a sus metadatos. Sin embargo, está dirigido a los administradores en lugar de a los usuarios finales, lo que sugerimos dirigir a la interfaz de usuario de Amundsen.

Apache Atlas es, hasta el momento, el único proxy en Amundsen que admite push y pull para recopilar metadatos: - Método de inserción aprovechando Apache Atlas Hive Hook. Es un detector de eventos que se ejecuta junto con Hive Metastore, que traduce los eventos de Hive Metastore en entidades de Apache Atlas y los envía al tema de Kafka, desde el cual Apache Atlas ingiere los datos mediante procesos internos. - Método de extracción aprovechando la integración de Amundsen Databuilder con Apache Atlas. Significa que los extractores disponibles en Databuilder se pueden usar para recopilar metadatos sobre sistemas externos (como PostgresMetadataExtractor) y enviarlos a Apache Atlas en una forma consumible por Amundsen.

https://www.amundsen.io/amundsen/metadata/

Conclusión.

Hasta ahora se ah logrado implementar el servicio de frontend de amundsen con podman y también el servicio de búsqueda, un pequeño problema es que en la documentación de Amundsen la información viene bastante amplia y no tiene instrucciones de cómo trabajar la herramienta a través de contenedores con podman, solo con docker, aunque es similar algunas cosas varían, con lo cual se sigue investigando también sobre los comandos y utilidades de podman, por otro lado debido a la investigación se ah descubierto que casi no existe información acerca de cómo implementar la herramienta amundsen con sus componentes integrales en videos o guías externas como YouTube, la mayoría de avance es gracias a la documentación que existe en su sitio oficial, con lo cuál se sigue investigando la misma documentación para explorar mejor la herramienta, también se ah descubierto que el servicio de metadatos puede ser integrado por un backend de Apache Atlas con lo cual también se pretende investigar como conectar un servicio de backend en Apache Atlas para que trabaje junto con el servicio de metadatos de Amundsen.

GRACIAS

Conociendo
México

800 111 46 34
www.inegi.org.mx
atencion.usuarios@inegi.org.mx

INEGI Informa